Now let's have a brief excerpt from a similar table for ASCII and play a game of "can you spot the problem"...

ASCII (Extremely Partial!)

Table 8-4. *ASCII codepoint*

Codepoint	Character	Byte value
47	/	0x2F

If you're struggling, have a look at the third row of the first table. Byte 0x2F lies within the range 0x21–0x7E; this means that UTF-1 continuation bytes collided with part of the 7-bit ASCII code page. In this case a UTF-1 encoded string could be misinterpreted as containing a forward slash character, which happens to be a special character for describing Unix file systems (it is the directory separator). UTF-1 went quickly and quietly to its grave and was replaced by the saner FSS-UTF (File System Safe UCS Transformation Format).

FSS-UTF specified that only values 0x00–0x7F (the 7-bit ASCII range) were valid for single-byte characters. Multi-byte characters could only be expressed by bytes 0x80 or greater; the valid encodings for one- to three-byte characters are given below.

FSS-UTF (Partial)

Table 8-5. *FSS-UTF codepoint ranges*

First codepoint	Last codepoint	Byte 0	Byte 1	Byte 3
U+0000	U+007F	00–7F		
U+0080	U+207F	80–BF	80–FF	
U+2080	U+8207F	C0–DF	80–FF	80–FF

The proposed FSS-UTF was modified further by Ken Thompson of Bell Labs (the story goes that Thompson's modifications were jotted down on a napkin over dinner in 1992) and, in 1993, became UTF-8. It was adopted by the Internet Engineering Task Force (IETF) in RFC 2277 (https://datatracker.ietf.org/doc/html/rfc2277).